

Building Autonomous Workflows with Sapphire

Practical patterns for local autonomy, controls, permissions, and bounded execution.

This manual describes the currently tested implementation in this repository and intentionally avoids claiming production-scale or distributed infrastructure.

It is a local runtime reference for learning, experimentation, scripting, and bounded automation work, not a verified multi-node training or cloud deployment platform.

Agent loop

A Sapphire agent can observe context, remember key information, plan workflows, select tools, execute actions, verify outcomes, and recover or re-plan when a step fails or yields an unexpected result.

Memory and planning

Working history and in-process key/value storage are available. SQLite-backed FTS5 memory is also available through the industrial utilities. These are local runtime features, not cloud-hosted vector systems.

Controls

Budgets cover steps, tool calls, elapsed time, and optional verifier or recovery callbacks. The goal is bounded execution and transparent failure handling rather than autonomous trust for unmanaged actions.

Not in scope

Parser-level agent declarations, event triggers, DAG orchestration, cluster scheduling, and full multi-agent recovery guarantees are not presented as completed production features.

Current status summary

Sapphire version 1.0.5 is a Python-based local runtime and language with a working interpreter, bundled tools, and optional integrations. It is appropriate for learning, local automation, and experimental AI workflows. It is not presented as a distributed training stack, a production security-certified platform, or a globally managed service.

Results vary by machine, Python version, OS configuration, library availability, and workload. The documentation here reflects what is actually exercised in this repository and what remains a roadmap item.

Version 1.0.5. Reproduce local measurements with `benchmarks/benchmark_runtime.py`. Values depend on the machine and environment.